Isolating the Pretraining Corpus in Molecular Property Prediction

Revised project plan

Gabriel Dupuis, gdupuis@mit.edu, graduate
Leonard Mauvernay, lmauvern@mit.edu, graduate

6.3930 | Check-in 1 | November 5, 2026

Project area: drug discovery, molecules, and proteins

1. Where the project stands

Our proposal asked whether the composition of the pretraining corpus affects downstream molecular property prediction. On November 2 we met Yuxin Xie, whose benchmarking work motivated that question. The question survived the meeting. Most of the rest changed.

She gave us an order of operations. Analyse the corpora before pretraining anything. Check whether coverage already explains the published results before assuming composition does. Find the smallest corpus that still produces a gain, then work at that size. Use one model and keep everything except the corpus fixed.

This document turns that advice into a project we can finish by December 1.

2. The benchmarking result we build on

2.1 What was compared

Yuxin Xie compared five pretrained models against Chemprop trained from scratch. Chemprop is a directed message-passing neural network that reads atoms and bonds [1]. The five models span the main families of self-supervised pretraining objective.

Model	Backbone	Pretraining objective	Corpus
CheMeleon [2]	D-MPNN	predict Mordred descriptors	1M PubChem
MolCLR [3]	GIN	contrastive, atom and bond masking	10M PubChem
MolFCL	CMPNN	fragment contrastive	250k ZINC15
MotiL	CMPNN	diffusion and motifs	250k ZINC15
MoLFormer [4]	Transformer	masked SMILES	1.1B

She benchmarked them on three families of data. MoleculeNet [5] is the legacy suite every paper reports, so her numbers line up with the published ones. The ADMET endpoints of the Therapeutics Data Commons [6] are newer, cover 22 endpoints and sit closer to what industry measures. Twelve further datasets came from BASF, Incyte and Novartis.

Each family carries its own split. MoleculeNet ran under two scaffold protocols. The first puts only singleton scaffolds in the test set, so every test scaffold is unseen during training. The second leaves non-singleton scaffold groups in the test set and is the protocol most papers use. TDC ran under the leaderboard's standard split. The pharmaceutical data ran under a time split, as in the original Chemprop work. There the model trains on historical compounds and predicts what arrived later, which is harder than a scaffold split because the new molecules differ both in scaffold and in how they were collected.

2.2 Why the comparison had to be rebuilt

She found three problems in published comparisons.

Papers rarely report their split protocol, so there is no guarantee that two models were measured the same way.

These models are sensitive to hyperparameter tuning and to ensembling. Searching a hundred configurations for one model and leaving the baseline at its defaults is a large unearned advantage. Her study gave every model the same tuning budget and the same ensembling.

Papers disagree on multi-target datasets. Some train a single multi-target model and others train separate single-target models.

2.3 What came out

The endpoints are few, so she used two complementary tests. A Friedman test followed by Nemenyi asks whether the pretrained models beat Chemprop across the endpoints of a dataset. CheMeleon ranked slightly ahead of Chemprop. The gap stayed inside the critical difference, so the two are not separable at this number of endpoints. A per-dataset test, as used in the CheMeleon paper, then marks each model best, tied with the best, or worse than the best.

- Gains concentrated on the classification endpoints and on QM7 and QM8. Those two carry computed labels rather than measured assay values, so they sit apart from the rest.
- Classification gains averaged under 3 ROC-AUC points, roughly 3 to 5 percent in relative terms, and were not consistently significant.
- MolCLR lost about 10 percent on one regression endpoint, so pretraining can also hurt.
- Larger gains appeared on quantum-mechanics regression and on some rank-based ADMET endpoints.
- On experimental regression, Chemprop was best or tied with the best pretrained model almost everywhere.
- On several pharmaceutical datasets, from a few hundred labels to tens of thousands, Chemprop was statistically better than the pretrained models.

Each model also has its own fragility. MoLFormer gains the most from hyperparameter tuning and is inconsistent at its defaults. MolCLR needs ensembling and loses badly on some endpoints without it. CheMeleon is the stable one.

2.4 The low-label regime

She subsampled public datasets and a Novartis assay down to roughly 60 to 100 labels. Chemprop with descriptor augmentation holds up at that size and comes close to the pretrained models. Chemprop without descriptors is the worst curve on the plot. So whether the baseline carries descriptors decides the comparison in exactly the regime where pretraining is supposed to pay off.

2.5 Her reading of the pattern

She attributes the pattern to the pretraining objective and calls this a hypothesis read off the results.

CheMeleon predicts chemical descriptors, which she treats as a generalisable signal. It never produces a large significant gain and it never collapses where the others do. MolCLR masks atoms and bond types and then asks the representation to stay the same. That builds in an insensitivity to small structural changes. Those changes often move a continuous property a long way, which is why the invariance hurts most on regression.

2.6 What surprised her

Three things. That there is no consistent gain at all, although Chemprop dates from 2019 and the five models are five or six years newer and published in strong journals. That the field compares models so inconsistently. And that on pharmaceutical data Chemprop is sometimes statistically better rather than merely not worse, at every dataset size from a few hundred labels upward.

3. What the field cannot currently separate

In those five models the backbone, the objective, the corpus and the corpus size all move together. No pair differs in one factor alone, so the benchmark cannot attribute the result to any of them.

Four explanations are live, and Yuxin says people move between them without deciding.

1. The pretraining objective is poorly defined.
2. The corpus does not cover enough chemical space.
3. The corpus composition is wrong.
4. The downstream head should be richer than a plain MLP. This one was raised at the MLPDS conference.

Her suggestion is to remove two of them by construction. Use CheMeleon alone, so the backbone and the objective are fixed. Keep the fine-tuning head fixed. Then vary only the pretraining set. What remains is a measurement of explanations 2 and 3, separated from the others.

That is the framing we adopt, and it is hers. Our contribution is a decomposition.

4. The two explanations we have to tell apart

Suppose a corpus change helps. Two mechanisms could produce that, and Yuxin was explicit that the project turns on separating them.

Coverage. A broader corpus puts more test molecules near something the model saw during pretraining. The gain should then track nearest-neighbour similarity from each test molecule to the corpus, and it should vanish for molecules far from everything in the corpus.

Information. Molecules that exist in nature carry regularities that transfer beyond their own region of chemical space. The gain should then survive at low similarity.

She expects natural products to be more diverse than a random PubChem or ZINC draw. Coverage is therefore the explanation we have to beat, and the one a careless experiment would report as a result.

She also gave us a preliminary observation worth testing at scale. Similarity between the test set and the labelled training set mattered less than similarity between the test set and the pretraining corpus.

5. Prior work that comes closest

Insilico and Incyte, MLPDS poster. They replaced CheMeleon's pretraining corpus with an internal assay library and beat public PubChem and ZINC. They also merged the internal library with a million public molecules. They attributed the result to chemical-space coverage, because the internal library spanned more subcategories. Yuxin is sending us the poster. This is the closest precedent and we read it before fixing the design.

ChemFM pretrained one model for the same number of steps on UniChem and on ZINC20, which is ten times larger. UniChem won 9 of 11 MoleculeNet tasks [10].

NaFM re-pretrained three published models on natural products and got mixed results on natural-product classification [11].

None of them holds architecture, objective and corpus size fixed while varying composition along a controlled axis. Yuxin suspects other groups have tried similar experiments, so we keep looking and will say what we find.

6. The plan

Five stages. The first two need no pretraining at all, so there is a real project even if the cluster is congested or the premise fails early. Each stage has a condition that would make us stop or change

course.

6.1 Stage A. Do the corpora differ?

Question. Does COCONUT occupy a different region of chemical space from a random ZINC sample, and do the benchmark molecules sit closer to one than to the other?

Method. Compute Morgan fingerprints and Mordred descriptors for three candidate corpora and for every benchmark test set, then report four things.

- A t-SNE map of the three corpora against the benchmark molecules.
- The distribution of nearest-neighbour Tanimoto similarity from each test molecule to each corpus.
- Bemis–Murcko scaffold counts and the overlap between corpora.
- Per-descriptor coverage ranges.

The three candidates are a random sample of ZINC20 [7], COCONUT 2.0 with its 695,133 unique natural-product structures [8], and a subset of ZINC selected by natural-product-likeness score [9]. That score measures structural resemblance to natural products. It says nothing about biological activity.

Why this comes first. Yuxin was emphatic on this point. If COCONUT covers roughly the space a ZINC sample covers, composition cannot act through coverage, and a pretraining run would tell us that slowly and expensively. She also asked us to look at the part of ZINC that resembles natural products, which is what the likeness-score corpus gives us.

Cost. CPU only. Mordred on 50,000 molecules takes about 25 core-hours.

Stop condition. Matching similarity distributions for COCONUT and ZINC would rule out the coverage mechanism. We would report that and move the project to the size question in Stage C.

6.2 Stage B. Does distance to the pretraining corpus predict where pretraining helps?

Question. On the models that already exist, does the benefit of pretraining concentrate on test molecules that resemble the pretraining corpus?

Method. The CheMeleon pretraining corpus is public. It is a 43.9 MB file of one million cleaned PubChem SMILES on Zenodo [12]. So we can measure, for every benchmark molecule, its similarity to the exact set CheMeleon saw. We fine-tune the released checkpoint and a scratch Chemprop on the same splits and seeds, then regress the per-molecule improvement on that similarity.

Why. This tests Yuxin's mechanism before we spend a GPU-hour on pretraining, and it extends her preliminary observation to a larger set. It also tells us which benchmarks are worth carrying forward, because a benchmark whose molecules all sit at the same distance from the corpus cannot show the effect.

Cost. Fine-tuning only. The CheMeleon authors fine-tuned on a laptop GPU.

6.3 Stage C. How much pretraining data do we actually need?

Question. What is the smallest corpus at which descriptor pretraining still beats scratch Chemprop?

Method. Re-pretrain CheMeleon on random PubChem subsets at 10k, 25k, 50k and 100k, with the released 1M model as the top of the curve. Fine-tune each on a fixed benchmark panel and plot the gain against corpus size.

Why. This is Yuxin's idea and it does two jobs. It fixes the operating size for Stage D, which is what makes the compute fit. It also answers a question she says the field has not settled. MoLFormer trains on 1.1 billion molecules and does not clearly beat CheMeleon at one million or MotiL at 250,000. She guesses 5,000 is too low and 50,000 is where the curve stabilises, and she has not run it. She added that even if composition turns out to do nothing, the size curve is worth reporting on its own.

Cost. About 150 GPU-hours. Section 8 gives the arithmetic.

6.4 Stage D. Does composition matter at a fixed size?

Question. At matched corpus size, does pretraining on natural products beat pretraining on a general library?

Method. Three corpora at the size chosen in Stage C, three seeds each.

Corpus	What it isolates
Random ZINC sample	the control
COCONUT natural products	composition and biological origin together
ZINC filtered by NP-likeness	composition without biological origin

Architecture, objective, batch size, epoch budget, splits and seeds stay identical across the three. We remove benchmark molecules from all three corpora first.

The third corpus separates molecules that look like natural products from molecules that are natural products. NaFM did not have that control.

Three seeds is not a luxury here. Yuxin's central result is that differences in this area sit inside the noise, so one run per corpus would tell us nothing.

Cost. About 225 GPU-hours.

6.5 Stage E. Where does any gain live?

Question. Is a gain confined to the chemistry its corpus covers?

Method. Bin the test molecules by nearest-neighbour similarity to each pretraining corpus and report the gain within each bin. Repeat under both MoleculeNet scaffold protocols, because the split changes how the test set sits relative to the corpus.

A gain that lives only in the high-similarity bin is coverage. A gain that survives in the low-similarity bin is information.

Cost. Fine-tuning and analysis only.

6.6 What we drop

Volatile compounds as a direction. Yuxin's objection decides it. Natural versus synthetic and volatile versus non-volatile are independent axes, and volatile molecules exist in COCONUT and in PubChem alike. Running both axes would mean a two-by-two we cannot afford in four weeks, and we would still owe an argument for wanting natural volatiles rather than synthetic ones. It stays available as a fourth corpus if Stage D finishes early. Her general warning was against trying many combinations at once.

7. Setting up the re-pretraining

7.1 The reference run

Corpus	1M PubChem molecules, cleaned
Targets	1613 Mordred descriptors, standard-scaled and Winsorized at 6 sigma
Model	D-MPNN, hidden 2048, depth 6, mean aggregation, 12.9M parameters
Training	batch 128, max 500 epochs, early stopping patience 50, split 70/20/10
Hardware	eight NVIDIA RTX 2080 Ti
Cost	about 1,000 GPU-hours, plus about 500 CPU-hours for the descriptors
Disk	about 40 GB for the 1M-molecule Zarr store

7.2 What we have to build

The pretraining script is `models/chemprop_mpnn/pretrain.py` in the CheMeleon repository [13]. It takes three arguments: a Zarr array of descriptors, an output directory, and a SMILES file whose rows match the Zarr rows. Swapping the corpus is supported directly, so no change to the model code is needed.

Three things are not supplied.

The script loads `feature_means_cached_<stem>.pt` and `feature_vars_cached_<stem>.pt` from the working directory and does not create them. We generate per-descriptor means and variances for each new corpus.

The descriptor pipeline runs through `models/features.py`, which takes a SMILES file and emits the cleaned list plus the Zarr array. We reuse it and apply the preprocessing the paper used. Drop SMILES longer than 150 characters, drop multi-fragment entries such as salts, and drop anything RDKit cannot parse.

The split inside the script is fixed at 70/20/10 with seed 1701. We keep it, so our runs stay comparable to the released model.

7.3 Two discrepancies to resolve first

The paper says 85 percent of descriptor targets are masked in the loss. The code sets `masking_ratio` to 0.15 and builds the mask as `rand_like(targets) < 0.15`. Whether a true entry means included or excluded depends on Chemprop's criterion. We read the source before changing anything.

The paper says the feedforward head has the same width as the message-passing block, which is 2048. The code passes `hidden_dim=1024`. We follow the code and say so.

7.4 The first job we run

Download the prepared 1M Zarr store from Zenodo and run a short pretraining job on it. That gives us seconds per epoch on cluster hardware, confirms the environment, and lets us compare our own 1M run against the released checkpoint. The compressed download is 3.0 GB and the uncompressed features need about 40 GB. We pin the environment to the versions the authors used, which the repository lists.

8. Compute

8.1 What Engaging gives us

Any MIT Kerberos ID registers itself by logging in at https://orcd-ood.mit.edu. No PI sponsorship is needed [14].

Partition	Resources	Max wall time	Base limit
`mit_normal`	CPU only	12 h	96 cores
`mit_normal_gpu`	GPU	6 h	2 GPUs, 32 cores
`mit_quicktest`	CPU only	15 min	48 cores
`mit_preemptable`	CPU and GPU	48 h	4 GPUs, 1024 cores

The L40S with 44 GB is the default GPU and there are four per node. The H200 with 140 GB is reserved for jobs that need the memory or float64. The preemptable partition additionally exposes A100 and H100 cards. Home storage is 200 GB, with 1 TB each of pool and scratch. Each space caps at one million files, which matters because a Zarr store is many small files.

The documentation publishes no GPU-hour quota. The limits that bind are wall time, two concurrent GPUs and fairshare priority. One page does warn that over-requesting CPUs per GPU "may use up your allocation", so some accounting exists even if its size is unpublished. We will ask ORCD what it is. The same page gives 6 hours for `mit_normal_gpu` in its table and 12 in its `sinfo` sample, so we will check on the login node.

8.2 What the runs cost

The reference run is 1,000 GPU-hours for a million molecules on eight RTX 2080 Ti cards. An L40S has 6.8 times the FP32 throughput of a 2080 Ti and 1.4 times the memory bandwidth. A D-MPNN mixes gather and scatter over graph edges with 2048-wide matrix products, so it will not reach the full throughput ratio. We assume a factor of three per GPU and treat 2.5 to 4 as the bracket. The pilot run in Section 7.4 measures it.

That puts the full million-molecule corpus at about 330 GPU-hours on one L40S.

Cost scales with molecules times epochs. Smaller corpora converge in more epochs, so the saving from subsampling is sublinear. The third column applies a factor of two as a pessimistic bound.

Corpus size	Linear estimate	With 2x epoch penalty	Six-hour jobs at 2 GPUs
10k	3 GPU-h	7	1
25k	8	17	2
50k	17	33	3
100k	33	66	6
250k	83	165	14
1M	330	330	28

8.3 What fits

A 50,000-molecule corpus fits in two or three six-hour jobs on two GPUs, or in a single preemptable job with room to spare. Stage D in full comes to about 225 GPU-hours, which is roughly nineteen jobs on `mit_normal_gpu`. Across two accounts that is a few days of calendar time if the queue cooperates. Stage C comes to about 150.

A single million-molecule run is 28 sequential jobs, each queueing on its own. That is weeks of calendar time for one corpus. We keep it as a late confirmation run on one corpus at most, and only on the preemptable partition. The six-hour wall time is the binding constraint on this project.

The CPU work is effectively free. Mordred on 50,000 molecules is about 25 core-hours, and one `mit_normal` job gives 96 cores for 12 hours. All three corpora calculate inside a single job. Fine-tuning is almost free as well, which is what lets us afford many seeds and both split protocols downstream.

8.4 How we maximise the free tier

1. Both of us register, so the team has four concurrent GPUs rather than two.
2. Every run checkpoints and resumes. This turns the six-hour wall into a chunk size. The script already uses Lightning checkpointing, so we add a checkpoint every fifteen minutes and resume from the last one.
3. Long runs go on `mit_preemptable` with `-requeue`. Four GPUs for 48 hours covers a whole corpus, and preemption then costs at most the time since the last checkpoint.
4. We request L40S and never H200. A 12.9M-parameter model at batch 128 fits easily, and the H200 nodes are contended by jobs that need the memory.
5. We keep CPUs per GPU low, because over-requesting can attach a second GPU to the job.
6. Descriptor calculation goes on `mit_normal`, not on a GPU node.
7. We train in `bf16-mixed`. The script sets no precision and defaults to fp32, and the 2080 Ti had no bf16 at all.
8. Zarr stores live in the 1 TB pool space rather than in home.
9. Batch scripts get debugged on `mit_quicktest` before a GPU slot is spent on them.

8.5 Fallback

If the queue makes full pretraining impossible, we continue pretraining from the released checkpoint on each corpus instead. Yuxin was clear about what that costs. It becomes hard to argue that what we

measured is a property of the pretraining set rather than of the starting weights. We would report it as the weaker experiment it is.

9. Evaluation protocol

Yuxin's warning is that evaluation decides the answer. Pretrained weights can carry a real benefit that the wrong fine-tuning setup hides.

- The same splits and the same seeds for every condition, so runs are paired.
- The same fine-tuning and hyperparameter budget for the pretrained model and for the baseline. Anything else reproduces the unearned advantage her study was built to remove.
- The same ensembling for every condition.
- Both MoleculeNet scaffold protocols, singleton and common, because the split changes how the test set relates to the pretraining corpus.
- A scratch Chemprop of the same size as the baseline, plus a random forest on Morgan fingerprints.
- Descriptors added to the Chemprop baseline wherever we study the low-label regime, since her curves show the comparison turns on that.
- Each benchmark's recommended metric.
- Spread across seeds reported rather than point estimates.
- Multi-target datasets handled one way throughout, and we state which.

10. Timeline and milestones

The write-up is due December 1 and presentations run from December 1 to 10. That leaves four weeks of work.

Week of Nov 3	Check-in 1. Stage A complete. Accounts on Engaging, 1M corpus downloaded, one pilot pretraining job timed.
Week of Nov 10	Stage B. Stage C launched.
Week of Nov 17	Check-in 2. Size curve and scratch baselines in hand.
Week of Nov 24	Stage D at three seeds. Stage E. Write-up.
Dec 1	Write-up due.
Dec 1 to 10	Final presentation.

At Check-in 1. We can say whether the three candidate corpora occupy different chemical space, how far the benchmark molecules sit from each, and what our compute allows.

At Check-in 2. We can say how small a pretraining corpus still produces a gain over scratch Chemprop, and we have the baseline the check-in asks for.

At the final presentation. We can say how large the composition effect is at matched size, whether it exceeds the spread across seeds, and whether it survives outside the chemistry of the corpus.

Success. A clear null counts. Yuxin said the size curve is worth reporting on its own, and the course says a careful study showing where a method fails makes a strong project.

11. Risks

The effect is smaller than the noise. Our corpora are far smaller than published ones and the differences in this area are small to begin with. We run three seeds, report spread, and run two corpora properly before adding the third.

Coverage explains everything. Selecting for natural-product likeness also shifts molecular size, polarity and scaffold diversity. Stage E is the test, and a coverage-only result is still a result worth presenting.

COCONUT is not clean. Its own authors note that misclassified synthetic entries remain in the database.

The queue. Fairshare could make the GPU work slow in calendar time even though the total is modest. Stages A and B are insulated from this, and the preemptable partition is the pressure valve.

Scope. Corpus effects depend on the model and the task, so what we find may not carry beyond descriptor pretraining on a small graph model. We will say so.

12. Open items

- Ask ORCD what the base GPU allocation is, and check the wall time on the login node.
- Read the Insilico and Incyte poster when Yuxin sends it, along with her slides.
- Search for other work that varies the pretraining corpus at fixed architecture. Yuxin suspects more exists.
- Resolve the masking and head-width discrepancies against the Chemprop source.
- Send Yuxin short updates as we go. She asked for them.

AI disclosure: we used Claude and ChatGPT for literature search, compute estimation, methodological critique and editing. The question came from class and the direction came from the meeting with Yuxin Xie.

References

[1] Heid et al. Chemprop: a machine learning package for chemical property prediction. *J. Chem. Inf. Model.* (2024).

[2] Burns et al. Deep learning foundation models for low-data regimes from classical molecular descriptors. *J. Chem. Inf. Model.* (2026).

[3] Wang et al. Molecular contrastive learning of representations via graph neural networks. *Nat. Mach. Intell.* (2022).

[4] Ross et al. Large-scale chemical language representations capture molecular structure and properties. *Nat. Mach. Intell.* (2022).

[5] Wu et al. MoleculeNet: a benchmark for molecular machine learning. *Chem. Sci.* (2018).

[6] Huang et al. Artificial intelligence foundation for therapeutic science. *Nat. Chem. Biol.* (2022).

[7] Irwin et al. ZINC20: a free ultralarge-scale chemical database for ligand discovery. *J. Chem. Inf. Model.* (2020).

[8] Chandrasekhar et al. COCONUT 2.0: a comprehensive overhaul and curation of the collection of open natural products database. *Nucleic Acids Res.* (2025).

[9] Ertl et al. Natural product-likeness score and its application for prioritization of compound libraries. *J. Chem. Inf. Model.* (2008).

[10] Cai et al. ChemFM as a scaling law guided foundation model pre-trained on informative chemicals. *Commun. Chem.* (2026).

[11] Ding et al. Pretraining a foundation model for small-molecule natural products. *Nat. Mach. Intell.* (2026).

[12] Zalte, Burns and Green. CheMeleon training data. Zenodo (2025).

[13] Burns. CheMeleon source code repository. GitHub (2025).

[14] MIT Office of Research Computing and Data. Engaging cluster documentation. Accessed November 5, 2026.